

RESEARCH ARTICLE | JUNE 24 2026

Performance of AI agents based on reasoning language models on ALD process optimization tasks ^{EP}

Special Collection: [Atomic Layer Deposition \(ALD\)](#)

Angel Yanguas-Gil 



J. Vac. Sci. Technol. A 44, 043410 (2026)

<https://doi.org/10.1116/6.0005313>



Articles You May Be Interested In

Plasma enhanced atomic layer deposition of $\text{SiN}_x\text{:H}$ and SiO_2

J. Vac. Sci. Technol. A (May 2011)

Temperature effects in oxidative molecular layer deposition (oMLD) of polypyrrole

J. Vac. Sci. Technol. A (March 2026)

HIDEN
ANALYTICAL

Trusted in Research
for over 40 years

Precision Quadrupole
Mass Spectrometers
for Vacuum, Gas, Plasma & Surface Science

Find Solutions for Your Research

Performance of AI agents based on reasoning language models on ALD process optimization tasks

Cite as: J. Vac. Sci. Technol. A **44**, 043410 (2026); doi: 10.1116/6.0005313

Submitted: 14 January 2026 · Accepted: 8 May 2026 ·

Published Online: 24 June 2026



Angel Yanguas-Gil^{a)} 

AFFILIATIONS

Applied Materials Division, Argonne National Laboratory, Lemont, Illinois 60439

Note: This paper is part of the Special Topic on Atomic Layer Deposition (ALD).

^{a)}Electronic mail: ayg@anl.gov

ABSTRACT

In this work, we explore the performance and behavior of AI agents based on reasoning large language models on atomic layer deposition (ALD) process optimization tasks. In these tasks, an agent has to iteratively explore a process configuration space to identify the optimal dose times for the precursor and the coreactant, generally without any prior knowledge about the process, including whether it is actually self-limited. The agent is meant to interact iteratively with an ALD reactor in a fully unsupervised way, receiving feedback on the results of the proposed experiments. We evaluate this agent using a simple model of an ALD tool that incorporates ALD processes with different self-limited surface reaction pathways as well as a nonself-limited component. Our results show that agents based on reasoning models like OpenAI's o3 and GPT5 consistently succeeded at completing this optimization task, with a performance on par or superior to that of previous machine learning approaches. However, we observed significant run-to-run variability due to the nondeterministic nature of the model's response and search strategy. In order to understand the logic followed by the reasoning model, we captured the reasoning language model's open response detailing the reasoning process. An analysis of the responses showed that the logic of the model was sound and that its reasoning was based on the notions of self-limited process and saturation expected in the case of ALD. However, the agent can sometimes be misled by its own prior choices when exploring the optimization space, which contributes to the variability of the results of the optimization process.

Published under an exclusive license by the AVS. <https://doi.org/10.1116/6.0005313>

I. INTRODUCTION

The field of generative AI has been yielding models with increasing levels of performance. One of the most significant recent breakthroughs has been the development of the so-called reasoning large language models (LLMs). These models, which include examples such as OpenAI's o1 and o3, Qwen/QwQ, and DeepSeek's open-weights R1,^{1,2} are significantly better than traditional LLMs at so-called reasoning tasks in areas such as math and coding.^{3,4} Many of the commercially available models as of early 2026 are hybrid models, directing complex queries to these more advanced models.

How to effectively leverage LLMs in the physical sciences is still an active area of research.⁵ One promising application is using LLMs to build AI agents that can interact with autonomous

materials synthesis platforms. However, our understanding of LLMs' capabilities in the context of thin film growth is still lacking. This is a problem shared across the physical sciences, where the development of general methodologies to evaluate the performance of models and agents in useful, practical contexts is still in the early stages of development.⁶

In this work, we explore the potential of AI agents based on reasoning LLMs in the context of atomic layer deposition. In particular, we focus on the optimization of a traditional AB-type ALD process comprising a precursor and a coreactant. This is a common task both for the characterization of processes based on new ALD precursors and when adapting existing processes to a specific ALD reactor. It is also an ideal model system to evaluate the capabilities of reasoning models that has clear outcomes: if the process is self-limited, its optimization involves finding dose times

for which the process is fully saturated (or nearly so) while minimizing the total deposition time. However, if the experimental evidence suggests that the process is not self-limited, the optimization process should be abandoned. It, therefore, goes beyond traditional optimization algorithms, where the implicit assumption is that the process is self-limited.

While the optimization of a self-limited process is open to subjective analysis, domain experts are good at evaluating saturation curves and understanding the implications of choosing a specific dose time. Indeed, there is a wealth of experimental works in the literature describing and optimizing new ALD processes, a task that is generally carried out by obtaining saturation curves for both the precursor and co-reactant.^{7–13} This gives us a good baseline of how many samples are required and the steps that a human expert would follow when carrying out this task. Moreover, prior works have explored this problem from a machine learning perspective, offering a way to compare the performance of reasoning large language models with respect to more classical approaches,¹⁴ at least when the process is assumed to be self-limited.

We, therefore, seek to answer the following two questions: (1) how good are AI agents based on reasoning models at process optimization tasks? (2) How do reasoning models actually reason about ALD process optimization? The first question is multifaceted, and it includes gauging the model's ability to identify the optimal conditions, analyzing its run-to-run variability, quantifying the number of samples required and the impact of initial conditions and noise. The second question requires having access to the model's reasoning process and ensure that the suggested additional experiments are consistent with this logic.

To answer these questions, we have analyzed the performance of AI agents interacting with a simulated ALD tool. We have expanded a functional model of an ALD reactor used in a prior work¹⁴ to incorporate both soft-saturating ALD processes and the presence of a nonself-limited component. Details of this model will be presented in Sec. II D. We have then evaluated the performance of AI agents that interact with this virtual tool to optimize a given ALD process. We have explored a set of conditions designed to capture the diversity of real ALD processes in the literature. Our AI agent relies on a reasoning LLM to drive the optimization process and operates using a two-step process: in the first step, the reasoning model is asked to respond with the next steps in the optimization process using an open ended response format. This allows us to capture the reasoning steps the model has followed in each iteration. In a second step, this open ended response format is subsequently transformed into a structured output that includes a determination on whether the process has been optimized and the suggested next set of conditions. This structured output was chosen to be consistent with standard tool use approaches in LLMs and our experimental ALD reactors.

II. METHODOLOGY

A. Reasoning large language models

Reasoning large language models are an evolution of traditional large language models that overcome some of LLMs' limitations. While a traditional LLM stochastically generates outputs based on inputs on a single pass, a reasoning large language

model has a more complex structure designed for problem decomposition and validation prior to the generation of the final output. A key technique used in reasoning language models is the so-called chain-of-thought process, which breaks down a problem into multiple independent steps.¹⁵ Due to these features, the training and finetuning of the reasoning language models is also different from conventional LLMs, using techniques such as reinforcement learning to promote logic reasoning.^{1,2}

In a prior work, we explored LLMs' ability to answer queries focused on atomic layer deposition.¹⁶ The most advanced model used in that work was GPT4o, which is a conventional LLM. In this work, we use a reasoning large language model as a building block to construct an agent capable of autonomously carrying out the optimization of an ALD process. In particular, we focus on two reasoning LLMs: o3, a pure reasoning model from OpenAI, and GPT5, a hybrid model where complex queries are processed using models with reasoning capabilities. However, the methodologies used in this work are general and model agnostic and could be used with any other commercial and open weight model. Other examples of reasoning models include Qwen/QwQ and DeepSeek R1.^{1,2}

B. AI agents based on LLMs

AI agents are autonomous systems capable of planning, reasoning, and accessing external tools to accomplish multistep tasks. While the concept of agents predates the advent of generative AI, the improvement in LLM capabilities has renewed the interest of leveraging these models for scientific applications.¹⁷ Two common ways of accessing external tools are through application programming interfaces (APIs),¹⁸ for instance through a JavaScript Object Notation (JSON) format used by models with so-called tool calling capabilities, or through standard interfaces such as model context protocol (MCP) servers.¹⁹ While AI agents have been used in related contexts involving materials and chemical synthesis,²⁰ our understanding of their ability to successfully carry out tasks relevant for ALD process optimization is still very limited. Addressing this gap is the purpose of this work.

C. AI agent for ALD process optimization

In this work, we have considered agents with the architecture shown in Fig. 1. It comprises a logic and a generative AI component, and it is designed to interact with an ALD reactor (in this case a simulated ALD reactor) in a way that is compatible with our experimental ALD tools. The logic component iteratively asks the generative AI component to identify the optimal dose times for the precursor and coreactant that leads to a saturated growth per cycle (GPC), request new experiments to be carried out by the ALD reactor, or quit the optimization process if it deems that the process is not self-limited. The logic component then simply processes the model response, either requesting additional growths to the simulated ALD reactor or terminating the optimization process.

Since one of the goals of this work is to gain insights on the strategies and chain of thought used by reasoning models during the optimization of ALD processes, the AI component carries out two consecutive calls to the LLM during each iteration: the first call provides the context and instruction to the model, including the information obtained during prior iteration steps (in this case

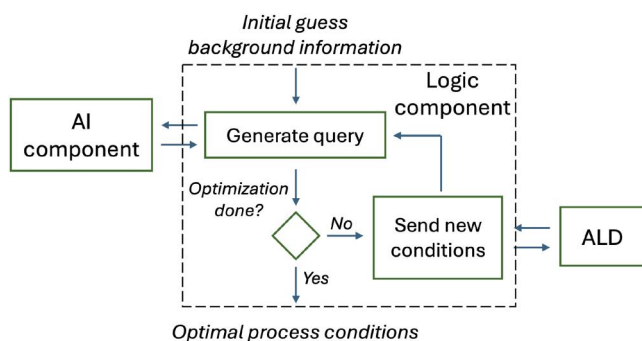


FIG. 1. Scheme of our AI agent for ALD process optimization: the logic component generates queries and process the response of the AI component, which uses a reasoning model to determine the strategy for optimization and request additional experiments. Based on the AI component's response, the logic component sends new conditions to the simulated ALD reactor.

growth per cycle for different precursor and coreactant dose times) and any prior information about the process (e.g., whether the precursor has a high or a low vapor pressure, or the expected growth per cycle for a known process). The LLM provides an open response on how to proceed, including suggestions for new conditions, and the determination of whether the process has been optimized and

whether it is actually self-limited. A second call to an LLM takes this open response and transforms it into a structured output containing a list of experimental conditions and two flags identifying if the process is optimized or not self-limited. The model is asked to generate this response in JSON format. Consequently, for each iteration, we have the open response with the model reasoning and a clear output that the procedural component can work with.

We consider two different agent variants: in our base agent, the reasoning model receives the prompt and the information gathered so far. In the memory variant, the reasoning language model also receives the model output for all prior iterations. This provides some continuity to the reasoning process across iterations.

Figure 2 shows an example of the original prompt passed to the reasoning model. Beyond a basic explanation of self-limited processes, there are no specific instructions about how to carry out the optimization process. Some variations of this prompt will be explored in Sec. III B. A full description of the prompts is provided in Appendix B.

D. ALD process modeling

In this work, we interface the AI agent with a simulated ALD process. Our surface kinetic model builds on prior works^{21,22} and has been used in the past to evaluate machine learning algorithms for ALD process optimization.¹⁴ We consider a self-limited surface kinetics comprising one or more surface reaction pathways, each

Sample of prompt to LLM

You are in charge of optimizing an atomic layer deposition process.

Atomic layer deposition (ALD) is a thin film technique where a given process is characterized by four times: the dose time for the precursor, the purge time for the precursor, the dose time for the coreactant, and the purge time for the coreactant.

ALD is self-limited: for long enough dose times the growth per cycle becomes saturated.

Your job is to determine if the process is already optimized based on the data provided and, if it is not saturated, provide some new experimental conditions to try.

Also, at some point if the dose times are too long and the growth rate keeps increasing, you may conclude that the process is not self-limited.

LLM response

```
{
  "optimized": false,
  "not ald": false,
  "steps": [
    {"precursor": 0.05, "coreactant": 1.0}, {"precursor": 0.10, "coreactant": 1.0}, {"precursor": 0.20, "coreactant": 1.0},
    {"precursor": 0.50, "coreactant": 1.0}, {"precursor": 1.0, "coreactant": 1.0}
  ]
}
```

FIG. 2. Sample of the prompt passed to the reasoning model as well as a sample of the structured JSON returned by the model during each iteration. The model response determines whether the agent concludes the optimization or continues requesting additional experiments.

characterized by a first order irreversible Langmuir kinetics for the precursor and the coreactant. For each of these pathways, the state of the surface can be expressed in terms of a fractional surface coverage θ_i that describes the fraction of surface sites that are occupied by dissociatively adsorbed precursor molecules. The total fractional surface coverage is the sum of each of the individual components multiplied by a scaling factor f_i that determines the relative prevalence of each reaction pathway,

$$\theta = \sum_i f_i \theta_i, \quad (1)$$

with $\sum_i f_i = 1$. With this approximation, in a fully self-limited process where the precursor and co-reactant are perfectly isolated by purges, the evolution of the surface coverage with time during the precursor dose is given by

$$\frac{d\theta_i}{dt} = k_1^i (1 - \theta_i), \quad (2)$$

Conversely, during the coreactant dose, the evolution of surface coverage with time is given by

$$\frac{d\theta_i}{dt} = -k_2^i \theta_i. \quad (3)$$

Here, the coefficients k_1^i and k_2^i are lumped parameters that incorporate the rate coefficient as well as the dependence with the precursor pressure.

The growth per cycle is, therefore, given by the net incorporation of precursor during the precursor dose step,

$$\text{GPC} = \text{GPC}_0 \int_0^{t_1} \sum_i f_i k_1^i (1 - \theta_i) dt, \quad (4)$$

where t_1 is the precursor dose time.

This model has been extended in the literature to cases where the precursor and coreactant are simultaneously present in the gas phase.^{22,23} If, for instance, we assume that there is a nonzero background pressure of the co-reactant during the precursor dose, Eq. (2) can be expressed as

$$\frac{d\theta_i}{dt} = k_1^i (1 - \theta_i) - k_c^i \theta_i. \quad (5)$$

Equation (5) provides a simple way of modeling systems with a nonself-limited component. This model leads to an asymptotic growth rate that, in the limit $t_1 \rightarrow \infty$, is given by

$$\text{GR}_0 = \text{GPC}_0 k_1^i \frac{k_c^i}{k_1^i + k_c^i}. \quad (6)$$

1. Fully self-limited ALD process

In the case of fully self-limited ALD processes, we assume that the precursor and co-reactant are perfectly isolated by sufficiently long purge times. With this approximation, it can be shown (see Appendix A) that the steady state GPC of a self-limited process obtained from solving Eqs. (2) and (3) is given by

$$\text{GPC} = \text{GPC}_0 \sum_i f_i \frac{(1 - e^{-k_1^i t_1})(1 - e^{-k_2^i t_2})}{1 - e^{-(k_1^i t_1 + k_2^i t_2)}}. \quad (7)$$

Despite its simplicity, Eq. (7) captures the saturation behavior of many ALD processes. For instance, we can incorporate soft-saturating ALD processes with a fast and a slow reaction component: in Fig. 3(a) we show the impact of having a second reaction pathway characterized by a saturation time that is five times slower as a function of the relative weight of the slower reaction pathway, f_2 .

2. ALD process with a CVD component

To model the presence of a nonself-limited CVD component with a growth rate GR_0 , we use Eq. (5). Note that, for the purpose of this work, we are aiming primarily at reproducing the phenomenological behavior that an agent would observe when optimizing an unknown ALD process that may have a nonself-limited component.

Using Eq. (6), we can compute the value of the coefficient k_c^i required to achieve an asymptotic CVD growth rate of GR_0 . As shown in Appendix A, the growth per cycle can be solved analytically for given precursor and co-reactant dose times t_1 and t_2 , resulting on the following expression:

$$\text{GPC} = \text{GR}_0 t_1 + \text{GPC}_0 \theta_{\text{lim}} (\theta_{\text{lim}} - \theta_0) (1 - e^{-(k_1 + k_c) t_1}), \quad (8)$$

where

$$\theta_0 = \frac{e^{-k_2 t_2} (1 - e^{-(k_1 + k_c) t_1})}{1 - e^{-(k_1 + k_c) t_1} e^{-k_2 t_2}}, \quad (9)$$

$$\theta_{\text{lim}} = \frac{k_1}{k_c + k_1}, \quad (10)$$

$$k_c = \frac{k_1 \text{GR}_0}{k_1 \text{GPC}_0 - \text{GR}_0}. \quad (11)$$

In Fig. 3(b), we show saturation curves with different degrees of nonself-limited components.

E. ALD optimization benchmark

In order to evaluate the agent's ability to optimize new ALD processes, we have created a benchmark comprising a few prototypical ALD processes shown in Table I. These processes provide a representative sample of commonly observed behaviors: from a saturation curve perspective, we want to cover four different regimes, which we refer to as fast/fast, fast/slow, slow/slow, and soft/fast processes. A fast half cycle is defined as one where saturation is reached for dose times of the order of 1 s. A slow process is a process requiring dose times of at least 5 s or longer. A soft process represents a saturation curve with both fast and slow components. We also considered two values of saturated growth per cycles, 1.0 and 0.3 Å/cycle. Finally, we explored the impact of nonself-limited components in a fast/fast ALD process.

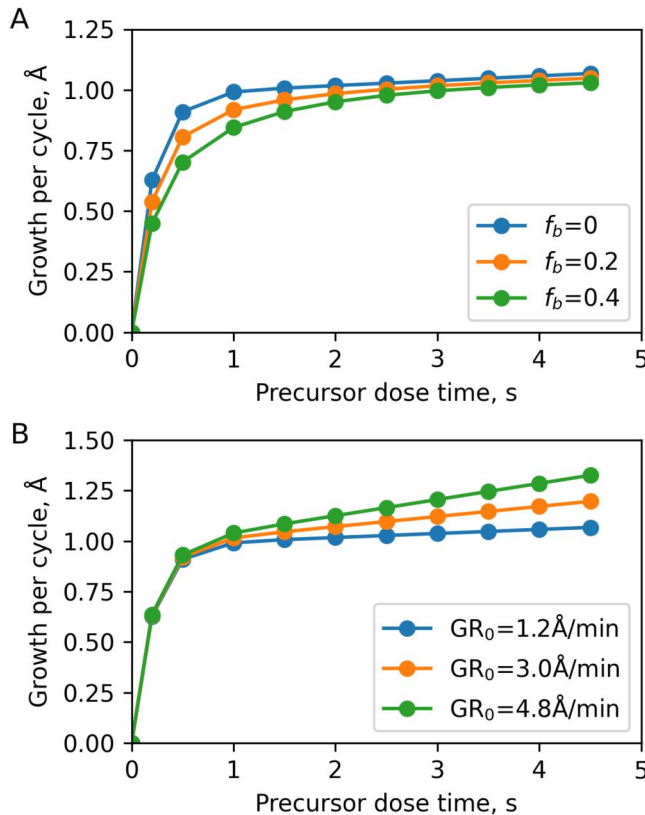


FIG. 3. Precursor saturation curves of simulated ALD processes used to evaluate the process optimization by AI agents: (a) Soft-saturating model, showing the impact of a second, lower reactivity reaction pathway ($k_1^a = 5 \text{ s}^{-1}$, $k_1^b = 1 \text{ s}^{-1}$, $k_2 = 4 \text{ s}^{-1}$); (b) Impact of CVD component for various values of the nonself-limited CVD component ($k_1 = 5 \text{ s}^{-1}$, $k_2 = 4 \text{ s}^{-1}$). In both cases, the coreactant dose time was set to $t_2 = 1 \text{ s}$.

III. RESULTS

A. Performance of the base agent

We have evaluated the base agent's ability to optimize each of the five ALD processes listed in Table I. We have tracked the AI agent's self-reported success on optimizing the ALD process, the selected precursor and coreactant dose times, the corresponding growth per cycle, the number of experiments (hereafter referred to

TABLE I. Main ALD processes used in this work.

Name	Model parameters
fast/fast, 1 Å/cycle	$k_1 = 5 \text{ s}^{-1}$, $k_2 = 4 \text{ s}^{-1}$, $GPC_0 = 1 \text{ Å}$
slow/slow, 1 Å/cycle	$k_1 = 1 \text{ s}^{-1}$, $k_2 = 1.2 \text{ s}^{-1}$, $GPC_0 = 1 \text{ Å}$
slow/fast, 1 Å/cycle	$k_1 = 1 \text{ s}^{-1}$, $k_2 = 4 \text{ s}^{-1}$, $GPC_0 = 1 \text{ Å}$
fast/fast, 0.3 Å/cycle	$k_1 = 5 \text{ s}^{-1}$, $k_2 = 4 \text{ s}^{-1}$, $GPC_0 = 0.3 \text{ Å}$
soft/fast, 1 Å/cycle	$k_1^a = 5 \text{ s}^{-1}$, $k_1^b = 1 \text{ s}^{-1}$, $f_b = 0.2$, $k_2 = 4 \text{ s}^{-1}$, $GPC_0 = 1 \text{ Å}$

as samples) required to carry out the optimization, and the number of iterations required for the agent to complete (or give up) the optimization process. We have first considered the case where the agent is provided an initial guess on the optimal dose times. We have evaluated two initial conditions: 0.2 and 2 s dose times for both the precursor and coreactant. These are referred to as (0.2 s, 0.2 s) and (2 s, 2 s). When optimizing either of these processes, the agent does not have any prior information on the expected growth per cycle of the ALD process. Therefore, we are evaluating the AI agent in the worst-case scenario of a completely new ALD process. For each condition, we have carried out ten independent runs.

Table II summarizes the key results: for each condition, we list the success percentage, defined the fraction of runs where the baseline agent concludes that the process has been successfully optimized; the optimal precursor and co-reactant dose times; the growth per cycle at the optimal conditions, both in Å/cycle and as a relative error ε , defined as the relative difference between the returned GPC and the saturated GPC, GPC_0 ,

$$\varepsilon = \frac{GPC_0 - GPC}{GPC_0}. \quad (12)$$

Finally, we consider the number of samples required to optimize the ALD process, the number of iterations in the algorithm, and the sum of the dose times.

We can make a few general observations: first, except for one run involving the soft-saturating ALD process (soft/fast), the agent reported successfully completing the process optimization in all the runs for all the ALD processes. When the initial guesses were far away from the saturated conditions, the agent provided optimized conditions whose growth per cycle was lower than the saturated growth per cycle. For instance, in the case of a slow/slow ALD process, optimizations started from an initial guess of (0.2 s, 0.2 s) resulted on an average GPC of 0.92 and 0.86 Å/cycle for the agents based on the o3 and GPT5 models. Similarly, optimizations from the same starting conditions of the soft/fast ALD process resulted in averages of 0.93 and 0.94 Å/cycle. In contrast, for both fast/fast processes with 1.0 Å/cycle and the slow/fast ALD process, the average GPC of the optimal conditions equaled or exceeded 95% that of the saturating GPC.

In Fig. 4, we represent the distribution of relative errors ε across all the five ALD processes and initial conditions explored in this section. This results in a total of 100 points for the agent based on the o3 and GPT5 models. The median relative error was 0.02 and the 75% percentile corresponded to a relative error of 0.05 for o3 and 0.04 for GPT5. To help visualize the quality of the selections, we plotted selected dose times returned by the agent on top of their corresponding saturation curves. Figure 5 shows one example corresponding to the 50th and 75th percentiles of processes optimized by an agent based on the o3 model. While the choice of a good set of conditions is somewhat subjective, it is clear that the AI agent tends to choose conditions that are somewhat undersaturated, particularly for R&D work where throughput is not as critical as in a production tool.

We observed significant run-to-run variability in the optimal conditions for the same ALD process and initial guess. For

TABLE II. Performance of the AI agent during the optimization of different ALD processes. In all cases, averages over ten independent runs are provided, with the number in parenthesis representing the standard deviation.

ALD process	Guess	Model	Success (%)	t_1 (s)	t_2 (s)	GPC (Å/cy)	Error, ε	# Samples	# Iter	$t_1 + t_2$ (s)
fast/fast 1.0 Å/cycle	0.2 s, 0.2 s	o3	100	0.86(0.15)	1.3(0.3)	0.97(0.02)	0.03	13(5)	5(1)	2.2(0.4)
		GPT5	100	0.87(0.11)	1.1(0.2)	0.97(0.02)	0.03	11(3)	4.2(1.0)	2.0(0.3)
	2 s, 2 s	o3	100	1.6(0.4)	1.6(0.4)	0.99(0.01)	0.01	9(3)	2.6(0.7)	3.2(0.9)
		GPT5	100	1.8(0.4)	1.8(0.3)	0.99(0.01)	0.01	4.7(1.1)	2.5(0.5)	3.6(0.7)
slow/slow 1.0 Å/cycle	0.2 s, 0.2 s	o3	100	3.2(1.0)	3.6(0.7)	0.92(0.08)	0.08	26(6)	8.0(1.7)	6.8(1.5)
		GPT5	100	2.5(0.9)	3.2(0.5)	0.86(0.11)	0.14	18(5)	6(2)	5.6(1.2)
	2 s, 2 s	o3	100	5.3(0.9)	4.8(0.9)	0.99(0.01)	0.01	10(2)	3.9(0.8)	10.1(1.4)
		GPT5	100	5.5(0.8)	4.7(0.6)	0.99(0.01)	0.01	8(2)	3.8(1.0)	10.2(1.2)
slow/fast 1.0 Å/cycle	0.2 s, 0.2 s	o3	100	3.8(0.6)	1.4(0.3)	0.97(0.01)	0.03	23(5)	6.7(1.2)	5.2(0.6)
		GPT5	100	3.4(0.8)	1.2(0.2)	0.95(0.04)	0.05	16(5)	7(2)	4.6(0.9)
	2 s, 2 s	o3	100	3.8(0.6)	1.0(0.4)	0.97(0.01)	0.03	9(2)	3.3(0.4)	5.8(0.9)
		GPT5	100	5.2(0.9)	1.8(0.3)	0.99(0.01)	0.01	7(2)	3.3(0.8)	7.0(0.9)
fast/fast 0.3 Å/cycle	0.2 s, 0.2 s	o3	100	0.79(0.16)	1.0(0.2)	0.29(0.01)	0.03	14(6)	5.0(1.2)	1.8(0.3)
		GPT5	100	0.79(0.16)	0.9(0.2)	0.28(0.01)	0.07	10(2)	4.3(0.6)	1.7(0.3)
	2 s, 2 s	o3	100	1.2(0.4)	1.8(0.3)	0.30(0)	0	7.3(1.6)	3.3(0.8)	3.0(0.5)
		GPT5	100	1.7(0.4)	1.95(0.15)	0.30(0)	0	5(2)	2.4(0.5)	3.6(0.6)
soft/fast	0.2 s, 0.2 s	o3	90	1.4(0.5)	1.3(0.3)	0.93(0.02)	0.07	15(5)	4.9(1.1)	2.6(0.8)
		GPT5	100	1.6(0.5)	1.2(0.3)	0.94(0.03)	0.06	10(2)	3.7(1.1)	2.8(0.6)
	2 s, 2 s	o3	100	2.8(0.8)	1.8(0.3)	0.98(0.01)	0.02	7(2)	3.3(0.9)	4.7(1.0)
		GPT5	100	3.3(0.6)	1.9(0.2)	0.99(0.01)	0.01	9(2)	3.2(0.9)	5.2(0.8)

instance, for the slow/slow ALD process, the standard deviation of the precursor and coreactant doses times optimized by an agent based on the o3 reasoning model from a (0.2 s, 0.2 s) initial guess had a standard deviation of 1 s, which is more than 30% of the average optimal dose time. The initial guess also had a big impact on the optimal dose times, with the optimization starting from a (0.2 s, 0.2 s) initial guess often resulting in significantly shorter dose times. This is shown in Fig. 6, where we represent the optimal dose times returned by the AI agent for the fast/fast 1 Å/cycle ALD process. The blue dots represent runs with a (2 s, 2 s) initial guess, while the red dots are those initiated with a (0.2 s, 0.2 s) guess. There is clearly a segregation between the two conditions for both the o3 and GPT5 models, with at least two runs started using the (0.2 s, 0.2 s) guess returning optimal dose times that lead to growth per cycles below 95% of the saturation value.

In terms of the number of experiments required to optimize an ALD process, for most of the conditions explored the average number falls between 10 and 15. Two exceptions were the slow/slow and slow/fast ALD processes starting from a (0.2 s, 0.2 s) guess, where the average number of samples for o3 exceeded 20. A significant number of the runs starting from (2 s, 2 s) were able to find optimal dose times with an average of fewer than 10 samples. This number is almost an order of magnitude lower than the number of samples required to optimize an ALD process using a gaussian process optimization approach described in one of our prior works.¹⁴ However, in that work, the machine learning algorithms had to optimize both the dose and purge times of an ALD

process. The number of experiments required by the AI agent is also comparable to the number of experimental points in works reporting experimental saturation curves for both the precursor and coreactant (e.g., Refs. 7–13). This indicates that, in terms of sample efficiency, the performance of agents based on reasoning language models is within range of that of human experts. However, as in the case of the optimal dose times, we also observed significant run-to-run variability, as evidenced by the large standard deviation in some of the entries in Table II.

From an algorithm's perspective, the number of iterations required to achieve the optimization of the fast/fast ALD process ranged between three and six or both models and starting guesses. In each iteration, the agent usually requested between two and four new experimental conditions. In Fig. 7 we show how the number of samples increases during an optimization process with the number of iterations. Each trace in Fig. 7 represents an independent run for the fast/fast, 1 Å/cycle model for the (0.2 s, 0.2 s) initial guess. There is a large run-to-run variability in the way the agent suggests new samples and the total number of iterations required. In Sec. III E we will explore in more detail the underlying strategy reported by the agent during each iteration.

B. Performance of the base agent without initial guess

The results obtained in Sec. III A show that the initial guess greatly impacts the performance of the optimization process, both

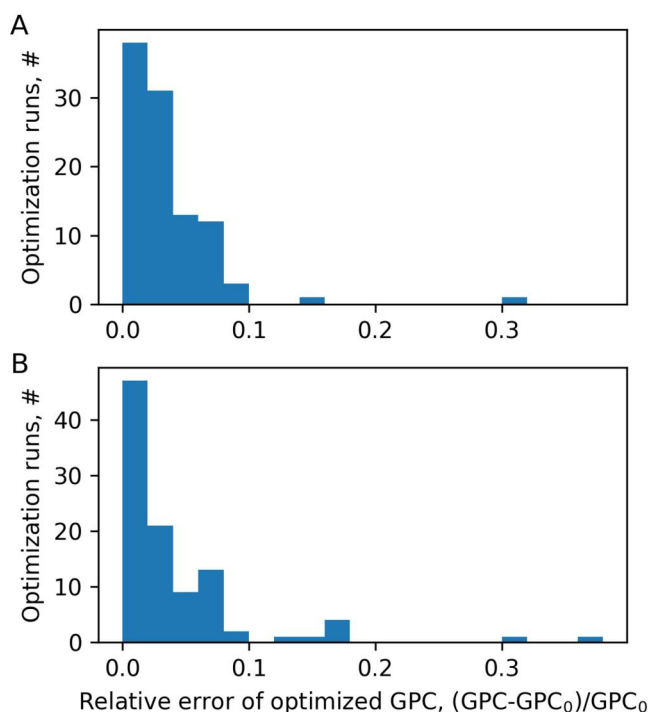


FIG. 4. Relative error between the growth per cycle resulting from the optimization process by the AI agent and the saturated GPC of the ALD process: (a) agent built on top of the o3 reasoning model; (b) agent built on top of the GPT5 reasoning model.

in terms of the agent's ability to identify saturated conditions and the number of samples required to carry out the optimization. This motivated us to explore how the model carries out the optimization without any initial guess and how other types of prior knowledge affected the optimization process when supplied to the model. We focused on the slow/fast ALD process with 1.0 Å/cycle.

We considered the following five scenarios: no prior information, a hint on the precursor pressure ("We expect the precursor to have a relatively low vapor pressure"), a hint on the growth per cycle ("We expect the saturated growth per cycle to be close to 1.0 Angstrom/cycle"), a hint on the initial dose time ("Start exploring dose times around 1 s"), and a scenario where we supplied both the pressure and the GPC hints. Since the differences between o3 and GPT5 are not substantial, we carried out this analysis on the agent based on o3, which is the pure reasoning model.

We summarize the average dose times, number of samples, and final growth per cycle in Table III. Except for one instance in the scenario where both the pressure and GPC hints were given to the agent ("Both"), the agent reported success at optimizing the ALD process in all the runs. The run deemed unsuccessful by the agent occurred because in the second step the agent incorrectly interpreted the open analysis of the reasoning model as a determination that the process was not self-limited, which terminated the optimization process. The most salient feature in Table III is the

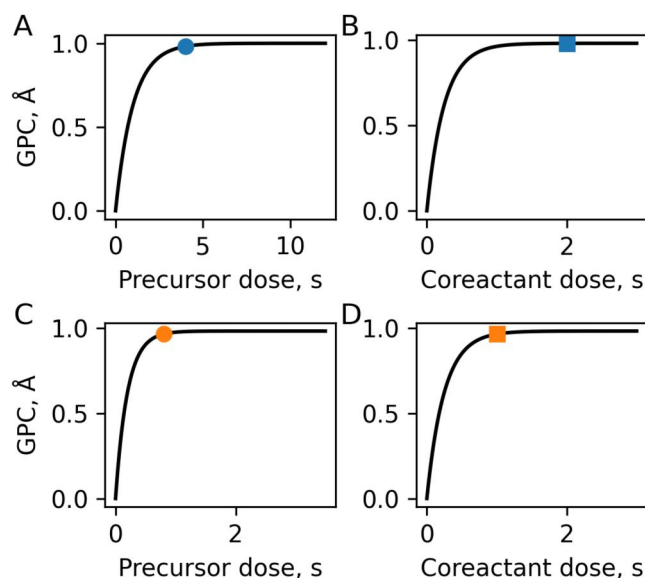


FIG. 5. Saturation curves and optimal times for two processes optimized by an agent built with the o3 reasoning model at the 50% and 75% percentile of relative error: (a) precursor saturation curve and (b) coreactant saturation curve for the slow/fast 1 Å/cycle ALD process (50th percentile) (c) precursor saturation curve and (d) coreactant saturation curve for the fast/fast 1 Å/cycle ALD process (75th percentile). Dots represent the optimal conditions returned by the AI agent.

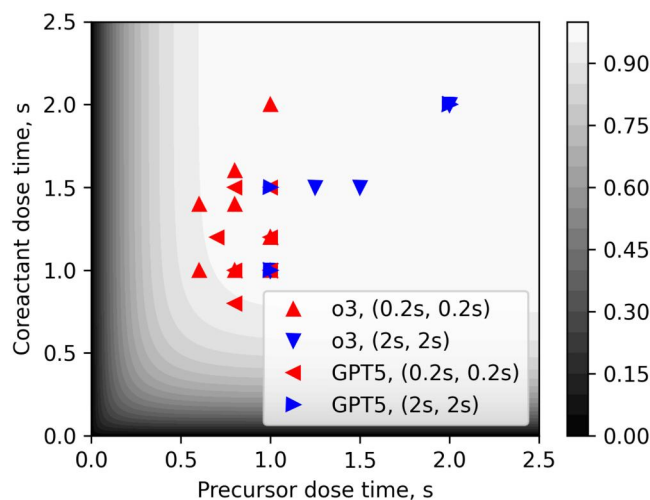


FIG. 6. Optimized precursor and coreactant dose times for AI agents based on o3 and GPT5 models for the fast/fast process with a GPC of 1 Å/cycle. In all cases the agents converged to an optimal solution, albeit with run-to-run variability as evidenced by the scatter in the plot. The greyscale contour plot represents the growth per cycle as a function of the precursor and coreactant dose time.

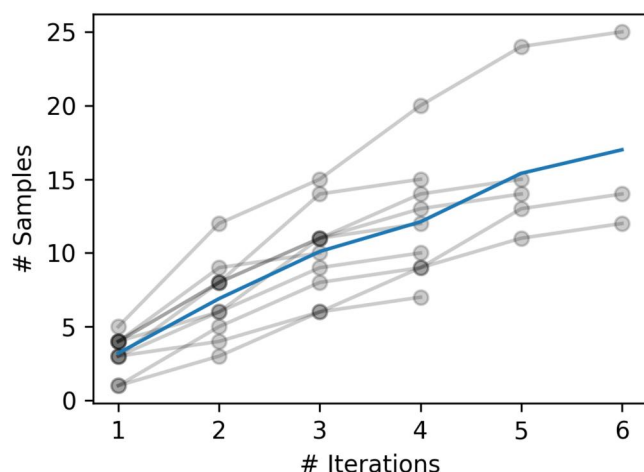


FIG. 7. Number of experiments requested as a function of the iteration number for ten independent optimization runs of the fast/fast 1 Å/cycle ALD process using an AI agent based on the o3 reasoning model for an (0.2 s, 0.2 s) initial condition. The blue line represents the average over all the runs.

large difference in the number of samples required to optimize the ALD process for the baseline scenario: in the absence of a guess, the agent required an average of 21 samples to optimize the ALD process. In contrast, when additional hints were supplied to the agent, the number decreases to the order of 14, which is a 33% decrease. This is consistent with the underlying reasoning model in the agent leveraging this additional information for its optimization strategy.

In Fig. 8, we show a scatterplot with the optimal conditions obtained by the AI agent under the different scenarios. As in the results in Sec. III A, we observe a significant spread on the optimal dose times, including a few runs where the optimal growth per cycle was 0.9 and 0.95 Å, which is significantly lower than the expected saturation value of 1 Å/cycle. We do not observe any trends in terms of the optimal dose times returned by the agent for any of the scenarios considered, with the average precursor and coreactant dose times and GPC being of the same order of those in Table III. This indicates that

TABLE III. Performance of the AI agent during the optimization of the slow/fast ALD process with no initial guess. In all cases, averages over ten independent runs are provided, with the number in parenthesis representing the standard deviation.

Scenario	Success (%)	t_1 (s)	t_2 (s)	GPC (Å/cy)	Error, ϵ	# Samples
Baseline	100	3.9(1.0)	1.3(0.3)	0.96(0.03)	0.04	21(6)
Pressure	100	3.9(0.9)	1.5(0.4)	0.96(0.03)	0.04	13(3)
GPC	100	4.0(0.7)	1.4(0.4)	0.97(0.02)	0.03	16(4)
Both	90	3.8(0.6)	1.4(0.3)	0.97(0.01)	0.03	14(4)
Dose	100	3.8(0.7)	1.3(0.3)	0.96(0.02)	0.04	14(4)

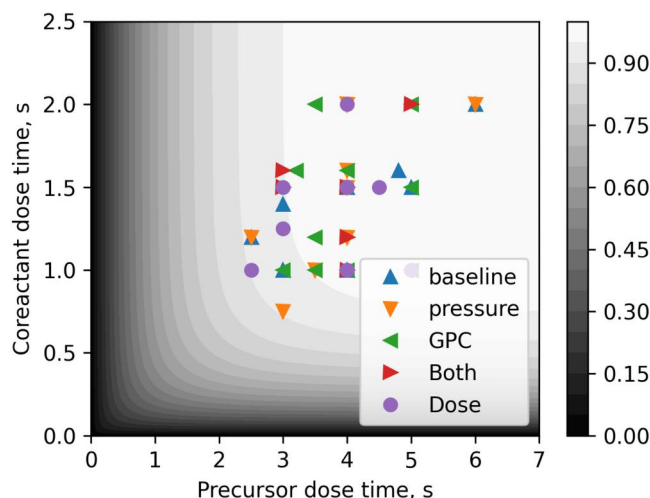


FIG. 8. Optimized precursor and coreactant dose times for an AI agent based on the o3 model optimizing the slow/fast ALD process without any initial guess. The different colors represent the different scenarios summarized in Table III. The grayscale contour plot represents the growth per cycle as a function of the precursor and coreactant dose time.

providing information to the agent primarily led to an increase in sample efficiency and not a significant change in the outcome of the optimization process.

Finally, we evaluated the agent's ability to identify the presence of a CVD component as it attempts the process optimization. We considered the fast/fast 1 Å/cycle ALD process and we added a CVD component as described in Sec. II D. This results in a linear dependence of the growth per cycle with precursor dose time [see Fig. 3(b)]. We asked the agent to optimize this process without any additional information. Then, we calculated the percentage of runs that ended with the agent concluding that the process is not fully self-limited.

We carried out this study for increasing magnitudes of the CVD component. For a CVD component of 3 Å/min, only 3 out of 10 runs concluded that the process was not self-limited. As the magnitude of the CVD component increases, the probability that the agent concludes that the process is not self-limited also increases, with 6 out of 10 runs and 5 out of 10 runs correctly identifying that the process was not self-limited for CVD components of 4.8 and 6 Å/min, respectively. The run-to-run variability observed in the fully self-limited ALD processes, therefore, extends to the determination of whether a new process is actually self-limited.

C. Comparison with other algorithms

In order to understand how the base agent's performance compares with more traditional machine learning approaches, we adapted two algorithms presented in Ref. 14 to the ALD processes explored in Sec. III A. In the context of ALD process optimization, there are two relevant metrics of performance: first is the ability to

TABLE IV. Performance of algorithms adapted from Ref. 14 on the five ALD processes introduced in Sec. III A.

Algorithm	ALD process	t_1 (s)	t_2 (s)	Error (ϵ)
Bayesian	fast/fast 1 Å/cy	0.85	1	0.03
	slow/slow 1 Å/cy	2.95	2.65	0.09
	slow/fast 1 Å/cy	2.95	1.15	0.06
	fast/fast 0.3 Å/cy	0.55	0.65	0.13
	soft/fast 1 Å/cy	1.2	1.05	0.07
Rule based 0.2 s, 0.2 s	fast/fast 1 Å/cy	0.73	0.83	0.06
	slow/slow 1 Å/cy	4.0	2.80	0.05
	slow/fast 1 Å/cy	4.0	0.76	0.06
	fast/fast 0.3 Å/cy	0.73	0.83	0.06
	soft/fast 1 Å/cy	1.2	0.83	0.09
Rule based 2 s, 2 s	fast/fast 1 Å/cy	0.92	1.02	0.03
	slow/slow 1 Å/cy	3.74	2.86	0.05
	slow/fast 1 Å/cy	3.74	1.02	0.04
	fast/fast 0.3 Å/cy	0.92	1.02	0.03
	soft/fast 1 Å/cy	1.8	1.02	0.05

achieve a growth per cycle that is as close as possible to the saturation growth per cycle, which we can evaluate with the relative error ϵ defined by Eq. (12). Second, it is the ability to select conditions that lead to a processing time that is as short as possible.

The first approach reported in Paulson *et al.* defined a universal cost function for an ALD process and then used a black-box Bayes-based search algorithm to explore the dose and purge time configuration space and identify the condition that minimizes this cost function. In Table IV, we show the performance of the Bayes-based process for the five conditions explored in Sec. III A. These values have been obtained by calculating the cost function using the algorithm defined in Paulson *et al.* and identifying the dose times that minimized that cost function. We used purge times of 5 s for both the precursor and the co-reactant. These values are consistent with those used in our experimental reactor.

The second approach explored in Ref. 14 used a rule-based algorithm that periodically switches between the dose times and the purge times to identify the optimal conditions. Like the baseline agent in Sec. III A, this method requires a starting value for the exploration. In Table IV, we show the performance of this algorithm for each of the initial guesses used for the baseline agent. The only difference between the algorithm used for Table IV and that of Paulson *et al.* is that here we are skipping any purge time optimization since this work only focuses on the dose times.

A comparison between the results for the baseline agent and the machine learning approaches shows that the baseline AI agent outperforms the cost function algorithm in terms of relative error ϵ . In the cost function scenario, the median error was 0.07 compared to 0.02 for the baseline agent using both o3 and GPT5. For the cost function method, the dose times are significantly shorter than those obtained by the baseline agent, as expected if it tends to converge to undersaturated conditions. This showcases the challenge of finding a universal cost function that is optimized for widely different ALD processes.

The baseline agent performance is on par of the rule-based method: while the median error is larger for the rule-based method (0.05), the distribution is more narrow, and the 75th percentile value is 0.06, which is very close to the value for o3, as shown in Sec. III A.

When we consider the case of the baseline agent without a guess, a comparison between the errors reported in Table IV and those in Table III shows that the error of the baseline agent was also of the order of or smaller than the error of both the Bayesian and rule-based algorithm for the optimization slow/fast ALD process considered in Sec. III B. This was true regardless of the hint provided to the agent.

Finally, in terms of sample efficiency, we computed the number of experiments required to optimize the ALD processes for the rule-based algorithm. The total number of samples ranged between 19 and 45, with the searches starting from a (2 s, 2 s) initial guess requiring fewer samples, as in the case of the baseline algorithm using initial guesses (Table II). The number of samples for the rule-based algorithm is larger than that required by the baseline agent in the absence of a guess. This indicates that, at least on average, the baseline agent was more sample efficient than the rule-based algorithm. We were not able to calculate the number of steps required for the cost function method without significant changes to the original implementation. However, it is worth mentioning that the cost function method requires computing gradients using finite difference methods, which increases the number of samples at least by a factor of two.¹⁴

D. Performance of agents with memory

To understand if the performance of the AI agent improved when the reasoning model is provided information on the model's reasoning used in past iterations, we carried out optimization runs on the conditions explored in Sec. III B using the memory variant of our AI agent with the o3 reasoning model. The results, summarized in Table V, did not show any major differences. We did observe an overall decrease in the number of experiments required when no hints were supplied and when an estimate of the dose time was supplied to the agent (baseline and dose scenarios in both Tables III and V).

In contrast, the average growth per cycle obtained at the end of the optimization process was slightly lower, which resulted in some runs providing optimal dose times that were not saturated (Fig. 9).

TABLE V. Performance of the AI agent with memory during the optimization of the slow/fast ALD process with no initial guess. The agent uses the o3 reasoning language model. In all cases, averages over ten independent runs are provided, with the number in parenthesis representing the standard deviation.

Scenario	Success (%)	t_1 (s)	t_2 (s)	GPC (Å/cy)	# Samples
Baseline	100	3.7(0.8)	1.0(0.2)	0.95(0.02)	18(5)
Pressure	100	3.9(1.1)	1.0(0.2)	0.94(0.04)	16(4)
GPC	100	3.7(0.7)	1.1(0.4)	0.94(0.06)	18(4)
Both	100	4.2(1.0)	1.1(0.2)	0.96(0.02)	18(2)
Dose	100	3.9(0.5)	1.1(0.3)	0.96(0.02)	13.6(1.4)

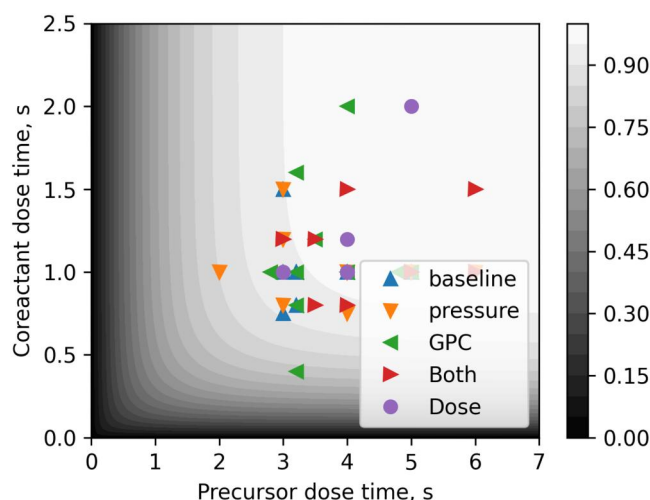


FIG. 9. Optimized precursor and coreactant dose times for an AI agent with memory based on the o3 model optimizing the slow/fast ALD process without any initial guess. The different colors represent the different scenarios summarized in Table V. The grayscale contour plot represents the growth per cycle as a function of the precursor and coreactant dose time.

This shows that passing the additional information to the underlying model to maintain consistency in the optimization process does not lead to significant improvements in the agent's performance.

E. Model search strategy and reasoning

For all the scenarios described above, we have compiled both the sequence of experiments requested per iteration and the

long-form responses from the reasoning language model supporting each request. We can use these data to explore in more detail the agent's optimization strategy. A simple way of visualizing this strategy is representing each experiment request as a scatterplot, where each point is colored according to the iteration number where it was requested.

We have analyzed these plots to identify a few salient strategies. We can then use the long-form responses to gain insights into the chain of thought that the reasoning model uses for each of the strategies. In Fig. 10, we present a representation of the experimental requests as an agent built on top of the o3 model as it attempts to optimize the fast/fast 1.0 Å/cycle process with an initial guess of (0.2 s, 0.2 s). In this representation, the model requests are colored by iteration. The final optimized condition is indicated by a square. Each of the plots in Fig. 10 represents one independent optimization attempt by the model. Note that some of the squares in Fig. 10 do not overlap with prior data points. This indicates that the optimal condition returned by the agent has not been experimentally validated.

The first salient feature is the lack of a unique strategy: in some instances, such as Figs. 10(a), 10(d), 10(f), and 10(g) the agent starts exploring regions near the initial guess and then moves to higher dose times. In other instances, such as Figs. 10(b), 10(h), 10(i), and 10(j) the optimization clearly moves along 1D saturation curves. Finally, the optimization runs captured in Figs. 10(c) and 10(e) present a hybrid strategy, where exploration takes place primarily along saturation curves, some of which were interrupted before moving into a different part of the configuration space. There is also significant variability in terms of whether the final proposal from the agent has been experimentally validated, as evidenced by the square not overlapping existing data points. For this specific case, 3 out of the 10 runs captured in

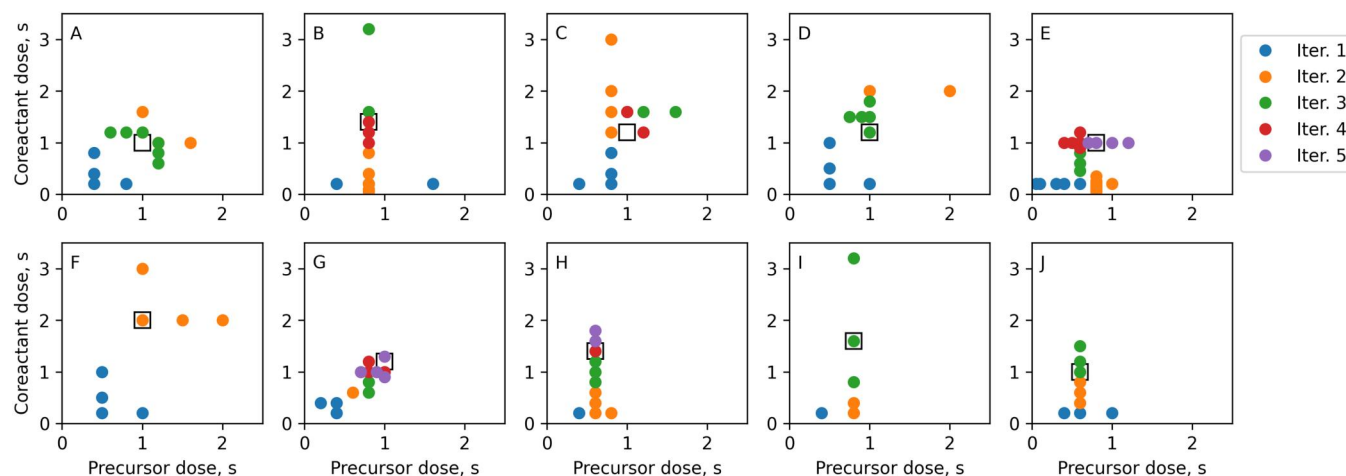


FIG. 10. Visualization of the search strategy of the baseline agent build on top of OpenAI's o3 reasoning model when optimizing a fast/fast ALD process with a GPC of 1 Å/cycle with an initial guess of (0.2 s, 0.2 s). Each color represents a request for additional data points made by the agent during a different iteration of the optimization process. Each plot represents an independent run: (a) Run 1; (b) Run 2; (c) Run 3; (d) Run 4; (e) Run 5; (f) Run 6; (g) Run 7; (h) Run 8; (i) Run 9; (j) Run 10.

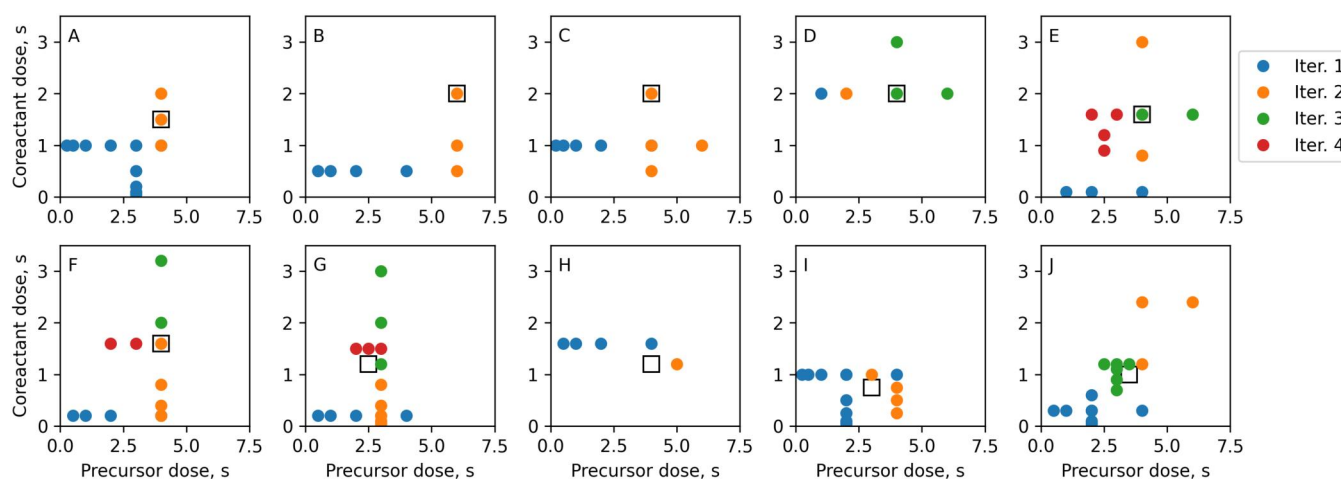


FIG. 11. Visualization of the search strategy of the baseline agent built on top of OpenAI's o3 reasoning model when optimizing a slow/fast ALD process with a GPC of 1 Å/cycle with no initial guess and with prior information on the precursor vapor pressure. Each plot represents an independent run: (a) Run 1; (b) Run 2; (c) Run 3; (d) Run 4; (e) Run 5; (f) Run 6; (g) Run 7; (h) Run 8; (i) Run 9; (j) Run 10.

Fig. 10 yielded optimal dose times that had not been experimentally validated.

The number of exploration points requested during the first iteration was small, approximately three, with the AI agent building its search strategy over subsequent iterations. This pattern of requesting a conservative number of experiments was observed across models and ALD processes. This makes the choice of conditions in later iterations dependent on the agent's prior choices. This may be one source of variability in the agent's strategy: small variations in the initial exploration can easily compound resulting in very different strategies.

In Fig. 11, we show a similar representation for the optimization of the slow/fast ALD process with 1 Å/cycle in the absence of an initial guess and with additional information on the precursor vapor pressure. The overall number of experiments required is similar than in Fig. 10. We can see the same overall strategies in this case. Also, 4 out of 10 optimal conditions have not been experimentally verified. However, a significant difference compared to optimizations starting from an externally supplied guess is the variability in the initial experiments requested by the agents (blue dots in Fig. 11). In terms of the number of requests, the majority of examples in Fig. 11 include 4 or more requests in the first iteration. Whenever the number of points was greater than one, the reasoning model favored primarily exploring the saturation behavior of the precursor [e.g., Figs. 11(b), 11(c), and 11(e)–11(h)]. In some of the runs, such as those represented in Figs. 11(a), 11(i), and 11(j), the reasoning model returns requests for saturation curves for both the precursor and the co-reactant. The dose time for the co-reactant during this first exploration showed a marked run-to-run variability as well.

When we look at the open response of the reasoning model, we observe that the model's responses consistently show that they are grounded in the physics of self-limited process. For instance,

the following is an excerpt of the agent's conclusion after the first iteration of an optimization process:

Outcome: If both precursor and coreactant show a clear plateau, the process is self-limited and optimized at those shortest plateau doses (likely ~0.40 s precursor, ~0.40 s coreactant). If GPC keeps increasing with dose without a plateau, the reaction is not self-limited and a different chemistry or temperature window must be considered.

Likewise, during the last iteration, the agent justifies ending the optimization process as follows:

With a generous coreactant dose (≥ 1 s) the GPC rises from 0.94 Å (0.6 s precursor) $\rightarrow$ 0.97 Å (0.8 s) $\rightarrow$ 0.99 Å (1.0 s) and then stays constant at 0.98–0.99 Å for 1.2 s and 1.6 s precursor. Therefore the precursor dose is saturated at about 1.0 s; anything longer offers no measurable gain.

The responses consistently showed this type of reasoning.

Finally, we can also look at the agent's reasoning when the process has an additional CVD component. This is a snapshot of the reasoning leading the model to conclude that the fast/fast 1 Å/cycle process with a CVD component of 5.0 Å/cycle is not self-limited:

When the coreactant is already saturated, the growth continues to climb monotonically from 1.03 Å/cycle (precursor 0.8 s) to 2.46 Å/cycle (precursor 15 s). No hint of a plateau is visible, even at 15 s. The rise is roughly proportional to dose, which is contrary to the self-limiting behaviour expected for ALD. Conclusion: The process is NOT optimized. The precursor half-cycle is clearly not saturated and, given the very

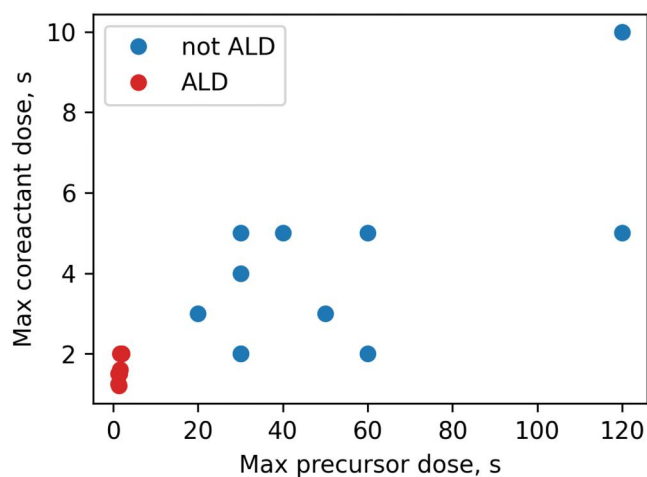


FIG. 12. Maximum precursor and dose times explored by the AI agent based on the o3 model for two fast/fast, 1 Å/cycle ALD processes with CVD components of 4.8 Å/min and 6.0 Å/min. Each point represents a separate run and they are color coded depending on whether the agent identifies the process as self-limited (red) or nonself-limited (blue).

long doses already explored, the chemistry is probably not self-limited under the present conditions.

This shows that the model's reasoning is largely correct. To gain further insight into how the agent's choices during the optimization can affect or bias their reasoning we focused on the ALD processes with a nonself-limited CVD component studied in Sec. III B. Figure 12 shows the largest dose times explored by the agent for independent runs of the agent built on o3 reasoning models for the fast/fast ALD process for two of the values for the CVD component explored in Sec. III B: 4.8 and 6.0 Å/min. Each point is a separate run and they are colored depending on whether the agent determined that the process was self-limited (blue) or nonself-limited (red).

Figure 12 shows a clear correlation between the range of dose times explored and whether the agent was able to correctly identify the process as a nonself-limited process. This confirms that the variability in the results does not seem to stem from reasoning issues, but from how the reasoning process is biased by the agent's own choices when exploring the design space.

IV. DISCUSSION

Agents based on reasoning models are capable of optimizing ALD processes without any prior knowledge of the growth per cycle and saturation times. In the majority of cases, they were capable of converging to reasonable dose times using fewer experiments than the rule-based algorithm in Ref. 14. The relative error of the optimized conditions returned by the algorithm with respect to the saturated GPC was also significantly lower than the Bayesian method. A deeper analysis showed a strong run-to-run variability in the agent's performance, particularly in terms of the

final choice of dose times. Moreover, agents consistently struggled to identify non self-limiting conditions without any additional priors such as the expected saturation GPC. This was true for both the o3 and GPT5 models.

The agents explored in this work rely on reasoning language models, more advanced models built on top of conventional LLMs that are capable of internally breaking down complex problems and use multiple calls to LLMs to work on individual steps and integrate the final results. When we analyzed the underlying reasoning process used by o3 and GPT5, the reasoning traces showed that the agents were using relevant concepts such as saturation plateaus, or making decisions based on the dependence of growth per cycle with dose times, that are well aligned with how human experts reason about ALD processes. However, we also saw a high degree of variability in the strategies used by the agents regardless of whether the agent had access to past reasoning steps or started with just a prior set of growth conditions at each iteration. For the case of the nonself-limited processes, we have seen that it was the search over the parameter space that biased the agent's determination of the self-limited nature of an ALD process.

We should note that we have purposely focused on the worst-case scenario for these agents: not only do the agents not have any prior information about the ALD process, but the prompt to the model in charge to the optimization process lacks details or instructions in terms of strategies to follow (see Appendix B). The performance reported should therefore be considered that of a baseline, minimum viable agent. As we have seen, the mere addition of hints to the agent's prompt reduced the number of samples required to optimize the process. Consequently, refinements in both the prompt and agent design will likely lead to significant improvements. In order to facilitate this exploration, we will make the benchmarks available as open source in a repository.

Based on these observations, how useful are AI agents based on reasoning models in practice? One of the most promising use cases for these agents is the optimization of new processes with real time feedback from *in situ* characterization techniques. The results show that in most conditions agents can explore the parameter space in a fully unsupervised way. If we used them exclusively for fully automated optimization tasks on self-limited processes, their advantages with respect to either rule-based algorithms or machine learning approaches are not significant. However, one unique capability is their ability to respond to different types of text-based requests.

From an experimental standpoint, the integration of these agents with ALD tools is straightforward. In a separate work, we describe how we have augmented an existing ALD tool with AI capabilities, including agents based on LLMs.²⁴ We were able to show that the AI component does not slow down the IO operations required to monitor and control an ALD reactor. One important consideration in experimental systems, particularly when real time feedback is used, is the lag between when a query or command is issued to the agent and the actual start of a new ALD growth. Experimentally, we have seen that this is dominated by the time the agent takes to process the request rather than the communication with the ALD reactor. In our specific case, this lag was always a few seconds per iteration, which makes agents based on reasoning models suitable for real time process optimizations

using *in situ* techniques. Details on the performance of the experimental system will be reported elsewhere.

V. CONCLUSIONS

In this work, we have demonstrated that AI agents based on reasoning large language models such as OpenAI's o3 and GPT5 can successfully optimize ALD processes in a fully autonomous manner without prior knowledge of the process parameters. These agents consistently identified optimal dose times, using fewer experiments than other algorithms used to solve similar tasks, typically requiring 10–15 samples for most processes. The performance of the agents are on par of superior to other algorithms described in the literature.

An analysis of the reasoning traces revealed that the models employ sound logic grounded in fundamental ALD concepts such as saturation plateaus and self-limiting behavior. However, we observed significant run-to-run variability in both the optimization strategies and the final dose times selected, with the agents' own choices during parameter space exploration influencing their conclusions. The agents also struggled to reliably identify nonself-limiting processes. While the baseline agent architecture presented here operates in a worst-case scenario with minimal prompting and no process-specific priors, the addition of even simple hints substantially improved sample efficiency, suggesting that refined prompt engineering and agent design could yield significant performance gains. The benchmarks and agents are available at: <https://github.com/aldsim/aldenv>.

ACKNOWLEDGMENTS

This research was based upon work supported by the Laboratory Directed Research and Development (LDRD) funding from Argonne National Laboratory, provided by the Director, Office of Science, of the U.S. Department of Energy under Contract No. DE-AC02-06CH11357.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Angel Yanguas-Gil: Conceptualization (equal); Data curation (equal); Formal analysis (equal); Funding acquisition (equal); Investigation (equal); Methodology (equal); Project administration (equal); Resources (equal); Software (equal); Supervision (equal); Validation (equal); Visualization (equal); Writing – original draft (equal); Writing – review & editing (equal).

APPENDIX A: DERIVATION OF ALD MODEL

If we define θ as the fraction of surface sites that have reacted with an ALD precursor, a simple irreversible Langmuir kinetics model established the following equation for the evolution of the

surface coverage as a function of time:

$$\frac{d\theta}{dt} = k_1(1 - \theta), \quad (\text{A1})$$

where the constant k_1 incorporates the rate coefficient as well as the dependence with the precursor pressure.

We can add a nonself-limited component by considering an effective concentration of co-reactant during the precursor dose, k_c , so that the kinetic model is now

$$\frac{d\theta}{dt} = k_1(1 - \theta) - k_c\theta. \quad (\text{A2})$$

During the co-reactant dose, we consider that the evolution of the surface coverage is given by

$$\frac{d\theta}{dt} = -k_2\theta, \quad (\text{A3})$$

where k_2 is a rate that depends on the co-reactant pressure.

With this simple model, an ALD process is determined by the rates k_1 , k_2 , and k_c .

If the fractional surface coverage at the beginning of the precursor dose is given by θ_0 , the evolution of the surface coverage as a function of time is obtained by integrating Eq. (A2),

$$\theta(t) = \theta_{\text{lim}}(1 - e^{-(k_1+k_c)t}) + \theta_0 e^{-(k_1+k_c)t}, \quad (\text{A4})$$

where

$$\theta_{\text{lim}} = \frac{k_1}{k_1 + k_c}. \quad (\text{A5})$$

Likewise, if we define the coverage at the end of the precursor dose time as θ_1 , the coverage at the end of the co-reactant dose time t_2 , θ_2 , is given by

$$\theta_2 = \theta_1 e^{-k_2 t_2}. \quad (\text{A6})$$

The growth per cycle is given by the total amount of precursor adsorbed during the precursor dose.

$$\text{GPC} = \text{GPC}_0 \int_0^{t_1} k_1(1 - \theta) dt. \quad (\text{A7})$$

This results in

$$\frac{\text{GPC}}{\text{GPC}_0} = k_1(1 - \theta_{\text{lim}})t + k_1 \frac{\theta_{\text{lim}} - \theta_0}{k_1 + k_c} (1 - e^{-(k_1+k_c)t_1}). \quad (\text{A8})$$

The growth per cycle has, therefore two components: a first, nonself-limited growth rate that is linearly dependent with the precursor dose time, and a second self-limited contribution. If we define

$$\text{GR}_0 = \text{GPC}_0 k_1(1 - \theta_{\text{lim}}), \quad (\text{A9})$$

we can express k_c as a function of a given nonself-limited component GR_0 in Å/s.

Finally, in an steady state ALD process, we have that

$$\theta_0 = \theta_2. \quad (\text{A10})$$

This allows us to solve for θ_0 using Eqs. (A4) and (A6), so that

$$\theta_0 = \frac{\theta_{\text{lim}}(1 - e^{-(k_1+k_c)t_1})e^{-k_2t_2}}{1 - e^{-(k_1+k_c)t_1-k_2t_2}}. \quad (\text{A11})$$

The pure ALD case is obtained for $k_c=0$, $\theta_{\text{lim}}=1$, which results in

$$\text{GPC} = \text{GPC}_0(1 - \theta_0)(1 - e^{-k_1t_1}), \quad (\text{A12})$$

and

$$\text{GPC} = \text{GPC}_0 \frac{(1 - e^{-k_1t_1})(1 - e^{-k_2t_2})}{1 - e^{-(k_1t_1+k_2t_2)}}. \quad (\text{A13})$$

The generalization to consider multiple independent reaction pathways is trivial.

APPENDIX B: PROMPTS USED FOR THE REASONING MODEL

Here, we provide verbatim the prompt used for the reasoning model during each iteration:

```
## Your job: process requests to operate an atomic layer deposition process.
```

```
You are in charge of optimizing an atomic layer deposition process.
```

```
Atomic layer deposition (ALD) is a thin film technique where a given process is characterized by four times: the dose time for the precursor, the purge time for the precursor, the dose time for the coreactant, and the purgetime for the coreactant.
```

```
ALD is self-limited: for long enough dose times the growth per cycle becomes saturated.
```

```
Your job is to determine if the process is already optimized based on the data provided and, if it is not saturated, provide some new experimental conditions to try.
```

```
Also, at some point if the dose times are too long and the growth rate keeps increasing, you may conclude that the process is not self-limited.
```

```
You only have to provide the dose times for the precursor and the coreactant. The purge times have already been optimized.
```

```
Remember that too long of a dose time is wasteful both in terms of precursor utilization and the process duration. Therefore, you have to find dose times for the precursor and co-reactant that are large enough to be saturated but not too so that there is significant waste.
```

```
Remember that all self-limited process start with a strong dependence of the growth per cycle with dose time until the growth becomes saturated.
```

```
Remember that a process may be saturated for the precursor dose, but not saturated for the
```

co-reactant. It is therefore paramount to check both.

The data provided will contain any useful prior information and a list of conditions listing the precursor dose time ("precursor"), coreactant dose time ("coreactant"), and the corresponding growth per cycle ("gpc")

In some cases no prior data will be available and you will have to provide initial guesses for the dose and purge times. In other cases you will receive a specific request in terms of the optimization strategy.

During each iteration, the prior growth conditions were appended to this prompt to generate the query that was passed to the model.

REFERENCES

- ¹A. Yang *et al.*, "Qwen3 technical report," [arXiv:2505.09388](#) [cs.CL] (2025).
- ²D. Guo *et al.*, *Nature* **645**, 633 (2025).
- ³M. Balunovic, J. Dekoninck, I. Petrov, N. Jovanovic, and M. Vechev, "Matharena: Evaluating LLMs on uncontaminated math competitions," [arXiv:2505.23281](#) [cs.AI] (2025).
- ⁴J. Zhang, C. Petrucci, K. Nikolić, and F. Tramèr, "Realmath: A continuous benchmark for evaluating language models on research-level mathematics," [arXiv:2505.12575](#) [cs.AI] (2025).
- ⁵S. Yu, N. Ran, and J. Liu, *Artificial Intelligence Chemistry* **2**, 100076 (2024).
- ⁶F. Cappello *et al.*, "EAIRA: Establishing a methodology for evaluating AI models as scientific research assistants," [arXiv:2502.20309](#) [cs.AI] (2025).
- ⁷T. Aaltonen, M. Ritala, and M. Leskelä, *Electrochem. Solid-State Lett.* **8**, C99 (2005).
- ⁸D. J. Comstock and J. W. Elam, *Chem. Mater.* **24**, 4011 (2012).
- ⁹D. Choudhury *et al.*, *J. Vac. Sci. Technol. A* **38**, 042407 (2020).
- ¹⁰J. Hämäläinen, T. Sajavaara, E. Puukilainen, M. Ritala, and M. Leskelä, *Chem. Mater.* **24**, 55 (2012).
- ¹¹J. Klaus, S. Ferro, and S. George, *Thin Solid Films* **360**, 145 (2000).
- ¹²K. B. Klepper, O. Nilsen, and H. Fjellvåg, *Thin Solid Films* **515**, 7772 (2007).
- ¹³T. Pilvi, E. Puukilainen, F. Munnik, M. Leskelä, and M. Ritala, *Chem. Vap. Depos.* **15**, 27 (2009).
- ¹⁴N. H. Paulson, A. Yanguas-Gil, O. Y. Abuomar, and J. W. Elam, *ACS Appl. Mater. Interfaces* **13**, 17022 (2021).
- ¹⁵J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," [arXiv:2201.11903](#) [cs.CL] (2023).
- ¹⁶A. Yanguas-Gil, M. T. Dearing, J. W. Elam, J. C. Jones, S. Kim, A. Mohammad, C. Thang Nguyen, and B. Sengupta, *J. Vac. Sci. Technol. A* **43**, 032406 (2025).
- ¹⁷O. N. Oliveira, L. Christino, M. C. F. Oliveira, and F. V. Pavlovich, *J. Chem. Inf. Model.* **63**, 7605 (2023).
- ¹⁸A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, *Nature Machine Intelligence* **6**, 525 (2024).
- ¹⁹H. Pan *et al.*, "Experiences with model context protocol servers for science and high performance computing," [arXiv:2508.18489](#) [cs.DC] (2025).
- ²⁰Y. Ruan *et al.*, *Nat. Commun.* **15**, 10160 (2024).
- ²¹A. Yanguas-Gil and J. W. Elam, *J. Vac. Sci. Technol. A* **32**, 031504 (2014).
- ²²A. Yanguas-Gil, J. A. Libera, and J. W. Elam, *J. Vac. Sci. Technol. A* **39**, 062404 (2021).
- ²³M. Reinke, Y. Kuzminykh, and P. Hoffmann, *Chem. Mater.* **27**, 1604 (2015).
- ²⁴A. Yanguas-Gil, J. C. Jones, S. Kim, C. T. Nguyen, and J. W. Elam, *Rev. Sci. Instrum.* **97**, 053903 (2026).